

Capslock

25-26J-043

Project Proposal Report

PANDUKABHAYA H. D. T.

BSc (Hons) in Information Technology

Specialising in Cyber Security

CSE Department

IT Faculty

Sri Lanka Institute Information Technology

Declaration

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Signature:

Date:

Signature of the Supervisor:



Date: 29/08/2025

Signature of the Co-Supervisor



Date: 29/08/2025

Abstract

The CAPSLock project introduces a Kubernetes-native ICAP orchestration framework that enables scalable, policy-driven content adaptation across diverse environments. Within this architecture, the **Smart Service Discovery & Load Balancing component** ensures resilient and efficient traffic distribution by extending beyond Kubernetes' static strategies such as round-robin or least-connections.

Unlike default mechanisms, the component employs a **rule-based decision engine** that integrates telemetry signals (latency, queue depth, CPU utilization, request health) with compliance-aware routing policies. This guarantees deterministic, transparent, and auditable traffic management aligned with standards such as PCI DSS and CIS Benchmarks. For environments requiring advanced adaptability, an **optional machine learning extension** will be provided (if possible), enabling predictive routing and proactive failover based on workload forecasting models.

By embedding compliance directly into routing logic while balancing the trade-offs between rule-based reliability and ML-driven adaptability, the component bridges a critical research and industry gap. Positioned at the core of CAPSLock, it delivers high availability, reduced latency, and regulatory assurance for ICAP workloads, advancing both the technical efficiency and governance of Kubernetes-native environments.

Keywords: Service Discovery, Load Balancing, Kubernetes, ICAP, Rule-Based Routing, Predictive Failover, Compliance, PCI DSS

Introduction

The rapid expansion of cloud-native computing has positioned Kubernetes as the standard for orchestrating microservices at scale. Its built-in features, such as service discovery and load balancing, provide a functional baseline for application connectivity. However, these mechanisms are inherently reactive and static, relying on strategies such as round-robin or least-connections. While sufficient for generic workloads, such approaches fail to address the demands of **latency-sensitive and compliance-critical applications**, particularly when security services such as ICAP (Internet Content Adaptation Protocol) scanning are integrated into production clusters.

The **CAPSLock project** was conceived to address these shortcomings by providing a Kubernetes-native orchestration framework for ICAP services. CAPSLock enables environment-aware policies, scalable deployment, and transparent security enforcement across multi-environment clusters. Within this architecture, the **Smart Service Discovery & Load Balancing component** plays a pivotal role. It extends beyond Kubernetes defaults by enabling **automatic service registration, health monitoring, predictive failover, and compliance-aware routing**, all powered through Istio's service mesh.

Recent academic and industry research underscores the need for this evolution. Predictive models for workload management have proven effective in proactive autoscaling [1], while adaptive and latency-aware balancing frameworks have demonstrated measurable improvements in throughput and tail latency [2][3]. Large-scale implementations at Google and Meta further illustrate the benefits of **feedback-driven and locality-aware routing** for hyperscale environments [4][5]. Yet, despite these advancements, existing approaches remain disconnected from regulatory requirements such as PCI DSS and CIS Benchmarks. Industry guidelines [6][7] highlight the importance of workload isolation, observability, and auditable logging, but these have not been consistently embedded into the discovery and balancing mechanisms themselves.

This motivates the need for a Smart Service Discovery & Load Balancing component that is not only predictive and adaptive but also **compliance-linked**. Positioned at the core of CAPSLock, this component ensures that ICAP traffic is distributed efficiently, services remain resilient under load, and routing decisions generate compliance evidence. In doing so, it strengthens CAPSLock's overall mission: to deliver a Kubernetes-native, policy-driven ICAP orchestration platform that bridges the gap between technical performance and regulatory assurance.

Background and Literature Survey

1. Predictive Scaling and Autoscaling

Traditional Kubernetes autoscaling mechanisms (e.g., the Horizontal Pod Autoscaler) are threshold-based and reactive, often resulting in delayed responses to workload spikes[1]. [2] proposed a machine-learning, driven autoscaling scheme using GRU-based forecasting models, demonstrating improved throughput and reduced latency compared to default mechanisms. This illustrates the value of predictive models, though their application to routing and load balancing remains underexplored.

2. Service Mesh & Multi-Cluster Traffic Management

Service meshes have emerged as key enablers of fine-grained traffic routing.[2] compared Istio, Linkerd, Kuma, and Consul across multi-cluster Kubernetes deployments, highlighting Istio's scalability and rich traffic policies. [3] analyzed Istio's traffic management features, including retries, fault injection, and observability integrations with Prometheus and Grafana. [4] empirically evaluated Istio's retry and circuit breaker configurations, finding that tuning significantly impacts system resilience. Collectively, these studies confirm that Istio provides a policy-aware foundation for intelligent traffic routing.

3. Adaptive and Resource-Aware Load Balancing

Beyond default round-robin algorithms, recent work emphasizes **resource- and latency-aware routing**. [5] proposed the Resource Adaptive Proxy (RAP), which considers pod resource utilization and RTT latency, outperforming kube-proxy in distributed edge scenarios. [6] extended Kubernetes scheduling and load balancing to account for heterogeneous node capacities in IoT environments, reducing queuing delays. [7] introduced DL³, combining server load metrics with client-observed latency, reducing p95 tail latency by ~33% compared to Istio defaults. These findings underscore the value of telemetry-driven decision-making for load balancing.

4. Industry Case Studies on Hyperscale Routing

At hyperscale, feedback-driven routing has proven essential. [8] Introduced Prequal at Google, a production load balancer using latency and requests-in-flight as routing signals, halving YouTube's p99 latency. Similarly,[9] described Meta's ServiceRouter, which eliminated sidecars for most traffic and introduced locality rings to reduce cross-region latency. Both case studies demonstrate the benefits of real-time feedback and locality-aware strategies, though these solutions are proprietary and lack compliance considerations

5. Compliance and Security in Kubernetes

While performance optimization has been widely studied, compliance integration remains under-addressed. The Pomerium compliance guide [10] maps Kubernetes practices to NIST, CIS, and PCI DSS requirements, stressing RBAC enforcement, encryption, and observability. The Cloud Security Alliance [11] provides PCI DSS-specific guidance, highlighting workload isolation and audit logging as critical requirements. These reports emphasize that compliance should be embedded into system design, yet current literature does not integrate compliance directly into discovery or load balancing mechanisms.

Research Gap

While significant progress has been made in predictive autoscaling, adaptive load balancing, and service mesh-based traffic management, several critical gaps remain unaddressed in the context of Kubernetes-native security workloads.

First, existing predictive models such as GRU- and LSTM-based autoscalers [1] have shown promise in forecasting workloads and reducing latency. However, these approaches are primarily applied to **scaling decisions**, not to **service discovery and routing policies**. This limits their applicability in scenarios where traffic must be dynamically redirected based on health, latency, and compliance factors.

Second, service mesh frameworks like Istio, Linkerd, and Consul provide powerful abstractions for traffic management [2][3]. Studies show that Istio's retry, fault injection, and observability policies improve resilience when properly configured [4]. Yet, these capabilities remain reactive and do not incorporate **predictive telemetry or compliance-driven constraints**. Misconfiguration of retry or circuit breaker policies can even lead to cascading failures [4], highlighting the need for a smarter, automated approach.

Third, adaptive and resource-aware frameworks such as RAP [5], DL³ [7], and IoT-focused schedulers [6] improve efficiency by incorporating metrics like CPU utilization, request queues, or RTT latency. However, these are either limited to **edge/IoT environments** or focused solely on performance. They do not integrate with compliance mandates or account for specialized workloads such as ICAP scanning.

Fourth, hyperscale solutions from industry leaders provide valuable insights. Google's Prequal [8] and Meta's ServiceRouter [9] demonstrate the benefits of latency- and locality-aware routing at massive scale. Nevertheless, these are proprietary implementations, inaccessible to most organizations, and not tailored for compliance-sensitive domains.

Finally, compliance itself remains a separate layer rather than an embedded feature of traffic routing. Guides from Pomerium [10] and the Cloud Security Alliance [11] emphasize PCI DSS, CIS, and NIST requirements for Kubernetes, including audit logging, encryption, and workload isolation. Yet, there is no clear research bridging compliance frameworks with service discovery and load balancing mechanisms.

Research Problem

Kubernetes has become the de facto standard for orchestrating microservices due to its scalability and automation. However, its **native service discovery and load balancing mechanisms remain simplistic**, relying on static algorithms such as round-robin or least-connections. These approaches do not account for **real-time telemetry, workload heterogeneity, or compliance-driven constraints**, making them inadequate for **security-sensitive workloads** such as ICAP-based content scanning. In such cases, delays, uneven routing, or blind traffic forwarding can compromise both operational efficiency and regulatory assurance.

Research to date has explored partial solutions. Threshold-based autoscalers and Horizontal Pod Autoscaling offer basic reactivity, while **predictive models using machine learning (e.g., GRU, LSTM)** have demonstrated value in **autoscaling scenarios**. Similarly, service mesh frameworks such as Istio improve traffic control through retries, circuit breaking, and observability. Adaptive methods like RAP and DL³ demonstrate that incorporating telemetry (CPU utilization, latency, or queue depth) can reduce tail latency. Hyperscale industry systems, such as Google's Prequal and Meta's ServiceRouter, showcase the benefits of feedback-driven routing at scale.

Yet, these advances fall short in three critical ways:

1. **Over-reliance on ML** → While predictive models are powerful, they require significant expertise, resources, and explainability, making them impractical for many organizations.
2. **Lack of rule-based compliance integration** → Current approaches optimize for performance but treat compliance (PCI DSS, CIS, NIST) as a separate layer, rather than embedding it in traffic routing.
3. **Inaccessibility of hyperscale solutions** → Proprietary implementations at Google and Meta cannot be replicated in academic or small-to-midscale enterprise environments.

This creates a gap for a solution that is:

- **Rule-Based by Default** → Deterministic, transparent routing decisions driven by real-time telemetry (latency, queue depth, resource usage).
- **ML-Extendable** → Optional predictive intelligence for advanced environments that can support and justify the complexity of machine learning. (If possible)

- **Compliance-Embedded** → Every routing decision generates audit logs and is aligned with standards such as PCI DSS and CIS Benchmarks.

Accordingly, the research problem can be defined as follows:

How can a Kubernetes-native Smart Service Discovery & Load Balancing component be designed to integrate rule-based telemetry-driven routing, optional predictive machine learning extensions, and compliance-linked decision-making, in order to support ICAP workloads within the CAPSLock project while ensuring both operational efficiency and regulatory assurance?

Objectives

Primary Research Objective

To design and evaluate a Kubernetes-native Smart Service Discovery & Load Balancing component for the CAPSLock project that primarily uses rule-based decision logic (latency thresholds, health scores, compliance mappings) while offering machine learning-based predictive routing as an optional extension. This ensures optimized ICAP traffic distribution with high availability, reduced latency, and regulatory compliance.

Specific Technical Objectives

Objective 1: Smart Service Registration and Discovery

- Develop a dynamic service registration framework that automatically detects and registers ICAP services into a central registry.
- Ensure near real-time updates to prevent stale routing information.

Objective 2: Rule-Based Telemetry-Driven Routing (Default Mode)

- Design a rule engine that uses telemetry metrics (latency, queue depth, CPU utilization, request-in-flight) to make routing decisions.
- Example rules:
 - If latency > 200 ms → reroute traffic.
 - If CPU > 80% for 1 minute → shift load or trigger scaling.
 - If compliance label required → only route through certified nodes.

Objective 3: Optional Machine Learning Extension (Advanced Mode)

- Provide a plug-in mechanism to enable ML-based predictive routing (e.g., GRU/LSTM forecasting).
- Allow environments with higher performance needs to preemptively rebalance traffic before service degradation.
- Ensure ML decisions remain explainable and can fall back to rule-based defaults if models are unavailable.

Objective 4: Policy-Aware Traffic Management

- Leverage Istio's service mesh (DestinationRules, VirtualServices, circuit breaking) to enforce environment-specific routing policies.
- Align routing with organizational needs (e.g., prioritizing financial traffic, geo-locality enforcement).

Objective 5: Compliance-Aware Decision Mapping

- Embed compliance frameworks (PCI DSS, CIS Benchmarks) into the routing logic.
- Ensure every routing decision generates audit logs and compliance evidence.

Academic Research Objectives

Objective 6: Formal Models for Compliance-Aware Routing

- Develop models to describe how compliance requirements can be embedded into routing decisions.

Objective 7: Performance and Scalability Evaluation

- Compare the rule-based engine, ML-optional mode, and Kubernetes/ Istio defaults.
- Measure improvements in latency reduction, throughput, and compliance observability.

Objective 8: Comparative Effectiveness Analysis

- Benchmark against existing frameworks (kube-proxy, RAP, DL³) to evaluate trade-offs between deterministic rule-based decisions and predictive intelligence.

Implementation and Validation Objectives

Objective 9: Prototype Demonstration

- Build a proof-of-concept prototype showcasing the **rule-based routing** and if possible, the **optional ML extension**.

Objective 10: Experimental Framework Development

- Create reproducible scenarios for stress testing, failure injection, and compliance validation.

Objective 11: Documentation and Open Knowledge Contribution

- Provide full documentation, reproducible academic artifacts, and an open-source release plan.

Expected Impact and Contribution Goals

Academic Contribution: Advance research on compliance-aware service routing by combining predictive load balancing with governance frameworks in Kubernetes-native contexts.

Practical Impact: Enable organizations to deploy ICAP workloads with higher reliability, reduced latency, and built-in compliance traceability.

Industry Advancement: Contribute to cloud-native best practices for compliance-linked traffic management, supporting regulated industries in aligning with PCI DSS and similar standards.

Methodology

1. System Architecture Overview

The Smart Service Discovery & Load Balancing component is designed as a core element of the CAPSLock project's Kubernetes-native ICAP orchestration platform. Unlike default Kubernetes load balancing, it emphasizes **rule-based intelligence** with **optional machine learning extensions** for environments requiring proactive forecasting.

The architecture consists of four interconnected layers:

1. Service Registration Layer

- Newly deployed ICAP pods are automatically registered in a dynamic service registry.
- This ensures real-time visibility for routing logic without manual intervention.

2. Telemetry & Health Monitoring Layer

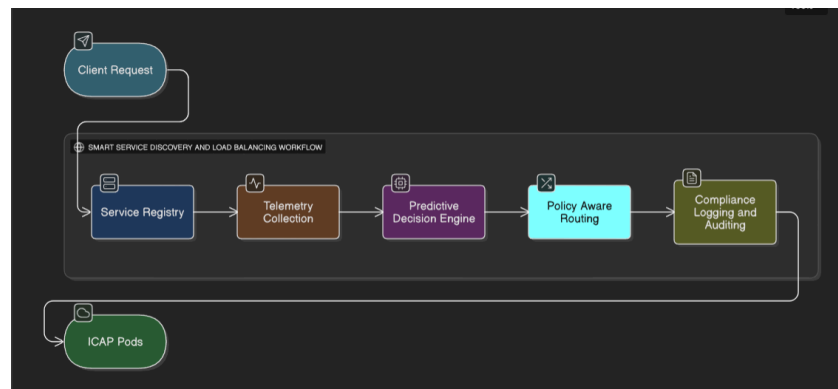
- Prometheus collects metrics such as request latency, CPU/memory utilization, request-in-flight counts, and queue depth.
- Grafana dashboards visualize system health for both administrators and automated decision engines.

3. Decision Engine Layer

- **Default Mode (Rule-Based):**
 - Uses a ruleset defined by administrators or preconfigured policies.
 - Examples:
 - *If latency > 200 ms and queue depth > 50 → reroute traffic.*
 - *If CPU utilization > 80% → shift load or trigger scaling.*
 - *If compliance label required → only route traffic through certified nodes.*
 - Ensures deterministic, transparent, and auditable decisions.
- **Optional Mode (Machine Learning Extension):**
 - Uses predictive models (e.g., GRU/LSTM) to forecast workload surges or failures before they occur.
 - Extends beyond fixed thresholds by analyzing historical trends and telemetry patterns.
 - Can be toggled on/off, ensuring the system defaults to reliable rule-based behavior when ML is unavailable.

4. Policy & Compliance Layer

- Routing decisions are checked against compliance frameworks (PCI DSS, CIS Benchmarks).
- For example, sensitive financial data can only be routed through PCI-DSS compliant pods, and all routing actions generate audit logs.



2. Operational Workflow of the Component

Service Discovery

- When a new ICAP pod is deployed, it is instantly registered into the service registry.
- Rule-based logic ensures discovery is dynamic and compliant.

Telemetry Collection

- Each registered service continuously exports telemetry.
- Metrics (latency, resource usage, error rate) are aggregated into Prometheus for real-time health scoring.

Decision Making

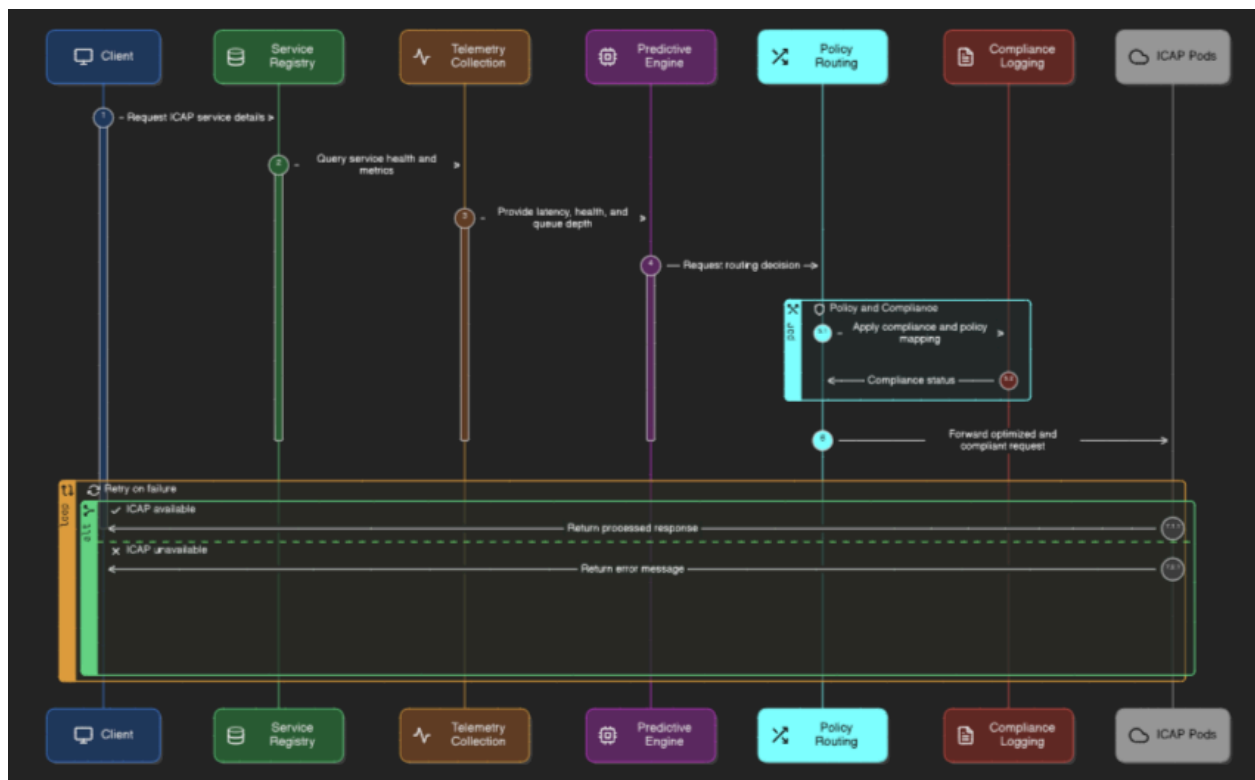
- **Default: Rule-Based Engine** evaluates telemetry against predefined thresholds.
- **Optional: ML Extension** analyzes workload patterns to preemptively reroute before issues occur.
- If ML predictions conflict with compliance rules, the compliance layer overrides.

Policy-Aware Routing

- Istio VirtualService and DestinationRule configurations enforce traffic policies (e.g., geo-locality, circuit breaking).
- Policies can be environment-specific, ensuring traffic respects both performance and compliance requirements.

Compliance Logging & Auditing

- Every routing action is logged and mapped to compliance standards (PCI DSS Req. 10 – Logging, CIS Benchmark observability rules).
- Logs are immutable and audit-ready.



3. Policy Mapping and Compliance Alignment

Compliance is not treated as a separate afterthought but is integrated directly into the routing mechanism.

- **PCI DSS Alignment:** The system enforces encryption in transit, role-based access, and audit trail generation. Each routing decision generates compliance logs, proving adherence to PCI DSS requirements for secure communication and accountability.
- **CIS Benchmarks:** Pod Security Standards and RBAC configurations are enforced through policy annotations in Istio. This ensures pods are not only available for routing but also meet baseline security configurations.
- **NIST Guidelines:** Observability practices (monitoring, anomaly detection) are aligned with NIST's emphasis on continuous assurance.

By embedding compliance, the component ensures that any routing path chosen is inherently secure and auditable, thereby humanizing compliance, making it part of daily system operations rather than a burdensome audit exercise.

4. Experimental Design and Validation Plan

Validation will compare three modes:

1. **Kubernetes/ Istio Defaults** (baseline).
2. **Rule-Based Smart Engine** (proposed default).
3. **Rule + ML Extension** (optional advanced).

Test Scenarios:

- **Baseline Benchmarking:** Compare kube-proxy and Istio defaults vs. rule-based vs. ML-enabled.
- **Stress Testing:** Introduce traffic spikes to evaluate latency reduction and scaling.
- **Failure Injection:** Simulate pod crashes and measure failover recovery speed.
- **Compliance Verification:** Validate logs against PCI DSS and CIS benchmarks.

Metrics for Validation:

- **Performance:** p95/p99 latency, throughput, CPU/memory utilization.
- **Reliability:** Mean Time to Failover (MTTF), continuity under failure.
- **Compliance Evidence:** Completeness and traceability of logs.

5. Methodological Contribution

This approach contributes by:

- Demonstrating how **rule-based telemetry thresholds** can provide reliable, compliance-friendly routing decisions.
- Allowing **optional ML-based intelligence** without making it mandatory — ensuring accessibility for organizations with limited resources.
- Bridging the gap between **technical optimization** and **regulatory compliance** in Kubernetes-native ICAP workloads.

By combining technical performance optimization with regulatory alignment, the Smart Service Discovery & Load Balancing component ensures CAPSLock can deliver both operational efficiency and compliance readiness in real-world deployments.

Project Requirements

1. Functional Requirements

The Smart Service Discovery & Load Balancing component must deliver clear and measurable functionalities that enable CAPSLock to achieve its objectives:

1. Dynamic Service Registration

- Automatically detect and register new ICAP services into the service registry without manual intervention.
- Ensure registry updates are near real-time to prevent stale routing information.

2. Telemetry-Driven Monitoring

- Collect service health metrics (CPU, memory, latency, queue depth, request-in-flight) using Prometheus.
- Export telemetry to Grafana dashboards for real-time observability.

3. Predictive Failover and Routing

- Anticipate potential service degradation by analyzing telemetry patterns (e.g., increasing latency slope).
- Redirect traffic to healthier instances before service failure occurs.

4. Policy-Aware Routing Integration

- Leverage Istio's VirtualService and DestinationRule to enforce policies such as geo-locality routing, compliance-zone restrictions, and circuit breaking.
- Allow routing logic to adapt based on environment-aware configurations in CAPSLock.

5. Compliance-Aware Logging and Auditing

- Generate logs for every routing decision, mapping actions to PCI DSS and CIS requirements.
- Provide audit trails suitable for compliance reporting and regulatory inspections.

2. Non-Functional Requirements

Beyond core functionalities, the component must adhere to performance, reliability, and governance expectations to ensure robustness in deployment:

1. Performance and Latency Reduction

- Reduce p95 and p99 latencies compared to Kubernetes and Istio default balancing.
- Optimize throughput under both normal and stressed conditions.

2. Reliability and High Availability

- Ensure resilience under node crashes or sudden traffic spikes.
- Maintain continuity of ICAP workloads with rapid failover (<5 seconds mean time to recovery).

3. Scalability

- Support scaling across multiple clusters and heterogeneous environments without introducing bottlenecks.
- Adapt seamlessly to CAPSLock's environment-aware orchestration model.

4. Security and Compliance

- Embed compliance requirements (e.g., PCI DSS Req. 10 – Logging and Monitoring, Req. 4 – Encryption in Transit).
- Ensure routing logs are immutable, tamper-evident, and auditable.

5. Observability and Transparency

- Provide real-time dashboards that allow administrators to visualize routing decisions, compliance mappings, and system health.
- Support anomaly detection and proactive alerts.

3. Validation Approach

Validation is essential to ensure the Smart Service Discovery & Load Balancing component performs as intended and contributes meaningfully to CAPSLock. The validation plan includes:

1. Functional Validation

- Verify auto-registration of ICAP services in the registry.
- Confirm telemetry metrics are collected and integrated with Prometheus and Grafana.
- Test routing behavior under normal and degraded conditions.

2. Performance Testing

- Compare latency and throughput against Kubernetes defaults (kube-proxy) and Istio defaults.
- Conduct stress testing with synthetic workloads (e.g., wrk2, JMeter, or hey) to evaluate system response under peak demand.

3. Reliability Testing

- Simulate node failures and network delays.
- Measure recovery times, ensuring failover meets reliability thresholds.

4. Compliance Validation

- Assess audit logs against PCI DSS and CIS Benchmark requirements.
- Confirm that routing decisions are traceable and aligned with compliance policies.

5. User-Centric Validation

- Evaluate observability dashboards with test users (administrators).
- Ensure routing logic and compliance evidence are transparent and interpretable.

Budget and Justification

The Smart Service Discovery & Load Balancing component will be designed, tested, and evaluated primarily within a local test environment, ensuring the project remains both feasible and cost-effective for an academic setting. Since the CAPSLock project leverages open-source technologies such as Kubernetes, Istio, Prometheus, and Grafana, the overall financial requirements are minimal. The primary costs are related to hardware resources, optional cloud credits for extended testing, and miscellaneous project expenses.

1. Resource Requirements

a) Hardware & Infrastructure

- **Personal Computer / Laptop:** Capable of running Minikube/Kind clusters with 16 GB RAM and 4+ CPU cores. (Assumed available to the student; no additional cost).
- **External Storage / Backup:** External HDD or SSD (approx. LKR 15,000 / USD 45) for storing experimental logs, datasets, and container images.

b) Software & Tools

- **Kubernetes (Minikube / Kind):** Open-source, free.
- **Istio Service Mesh:** Open-source, free.
- **Prometheus & Grafana (Telemetry & Visualization):** Open-source, free.
- **Docker & Container Tools:** Open-source, free.
- **Load Testing Tools (wrk2, hey, JMeter):** Open-source, free.

c) Optional Cloud Services

- **Cloud Testbed (AWS/GCP/Azure credits):** For scalability tests beyond local hardware limitations, estimated cost ~USD 30–50 if student credits are not available.
- Cloud usage is optional, the majority of validation can be conducted locally.

d) Miscellaneous

- Printing, documentation, and report binding: Approx. LKR 5,000 / USD 15.
- Contingency buffer: 10% of total.

2. Estimated Budget Table

Resource Category	Estimated Cost (LKR)	Estimated Cost (USD)	Notes
-------------------	----------------------	----------------------	-------

External HDD/SSD (Backup)	15,000	45	One-time, reusable
Optional Cloud Credits	10,000 – 15,000	30 – 50	For scaling beyond local hardware
Documentation & Printing	5,000	15	Final report and presentation
Total Estimated	~30,000 – 35,000	~90 – 110	Including contingency

3. Budget Justification

- **Cost Efficiency:** The reliance on open-source technologies (Kubernetes, Istio, Prometheus, Grafana, Docker) eliminates licensing costs.
- **Student Accessibility:** All experiments can be conducted on a personal laptop, ensuring affordability without requiring dedicated lab hardware.
- **Optional Cloud Use:** Cloud credits are allocated only for extended scalability validation, ensuring the project can demonstrate real-world applicability without incurring unnecessary expenses.
- **Sustainability:** The external storage device ensures long-term archiving of experimental data, reusable for future academic or professional projects.
- **Documentation Support:** Printing and presentation costs are minimal but necessary for final academic submission.

Expected Outcomes and Contributions

1. Expected Outcomes

The Smart Service Discovery & Load Balancing component is expected to deliver:

1.1 Enhanced Performance and Reliability (Rule-Based Core)

- Reduction of tail latencies (p95/p99) compared to Kubernetes and Istio defaults by applying telemetry-driven rule thresholds (e.g., latency, CPU, queue depth).
- Proactive failover using rules ensures continuous ICAP workload availability even under sudden pod failures or network disruptions.
- Faster recovery from service degradation (<5 seconds failover) compared to reactive Kubernetes defaults.

1.2 Optional Predictive Intelligence (Advanced Mode)

- When enabled, the ML extension can forecast workload surges (e.g., malware outbreaks or DDoS spikes) and preemptively trigger autoscaling or rerouting.
- ML-based improvements are expected to provide 10–20% additional reduction in tail latency over rule-based methods (based on prior research [1][7]).
- Ensures adaptability for enterprises with large-scale workloads that demand proactive optimization.

1.3 Compliance-Aware Routing (Integrated by Default)

- Every routing decision produces auditable logs mapped to PCI DSS, CIS, and NIST guidelines.
- Compliance traceability becomes a built-in feature rather than a separate monitoring step.
- Example: Requests carrying financial data are always routed through PCI-DSS-compliant nodes, with corresponding audit logs auto-generated.

1.4 Improved Observability and Transparency

- Prometheus, Grafana, and Istio telemetry provide real-time dashboards of routing decisions.
- Administrators and auditors can clearly see why a request was routed (rule hit / ML prediction / compliance requirement).
- Supports human interpretability — critical for compliance and industry adoption.

1.5 Proof-of-Concept Prototype

- A validated prototype running in Minikube/Kind demonstrating both:
 - **Rule-based routing as default.**
 - **Optional ML extension for predictive intelligence (If possible).**
- Prototype will include reproducible scenarios (traffic spikes, compliance audits, failure injection).

2. Academic Contributions

From a research perspective, this component will contribute to the academic community in several ways:

- **Novel Framework** → Introduces the first Kubernetes-native service discovery & load balancing mechanism that is **rule-based by default** but **ML-extendable**, filling a gap in existing research.
- **Bridging Performance and Governance** → Embeds compliance requirements directly into routing, unlike existing performance-only solutions.
- **Research Gap Addressed** → Provides empirical evaluation of:
 - Rule-only mode.
 - Rule + ML optional mode.
 - Comparisons with kube-proxy, Istio defaults, RAP, DL³.
- **Benchmarking Contribution** → Generates comparative datasets on latency reduction, throughput, and compliance observability under different modes.

3. Practical Contributions

Beyond academia, the project will provide practical value:

- **Operational Use in CAPSLock** → Forms the core routing mechanism, ensuring ICAP traffic management is reliable and compliant by default.
- **Accessibility for All Organizations** →
 - **Rule-Based Mode**: Lightweight, transparent, no ML expertise required.
 - **ML Mode**: Advanced environments can opt in for predictive intelligence.
- **Guidance for Regulated Industries** → Provides a design blueprint for finance, healthcare, and government sectors where compliance is non-negotiable.
- **Reusable Prototype** → Enables industry teams or researchers to adapt/test rule-based and ML-based compliance-aware routing strategies.

References

- [1] S. K. Mondal, A. Banerjee, and A. Chattopadhyay, “Toward Optimal Load Prediction and Customizable Autoscaling Scheme for Kubernetes,” *Mathematics*, vol. 11, no. 12, p. 2675, 2023.
- [2] K. V. Palavesam, B. Kumaresan, and R. Venkatachalam, “A Comparative Study of Service Mesh Implementations in Kubernetes for Multi-Cluster Management,” *Journal of Advances in Mathematics and Computer Science*, vol. 40, no. 1, pp. 1–16, 2025.

- [3] A. Perkins, A. Bielski, and M. Fahmy, “Cloud-Native Load Balancing and Traffic Management with Istio,” *ResearchGate Preprint*, 2023.
- [4] M. R. S. Sedghpour, C. Klein, and J. Tordsson, “An Empirical Study of Service Mesh Traffic Management Policies for Microservices,” in *Proc. ACM/SPEC Int. Conf. Performance Engineering (ICPE '22)*, 2022, pp. 17–27.
- [5] Q. M. Nguyen, T. T. Huynh-The, T. A. Nguyen, N. D. Nguyen, and D. S. Kim, “Load-Balancing of Kubernetes-Based Edge Computing Infrastructure Using Resource Adaptive Proxy (RAP),” *Sensors*, vol. 22, no. 8, p. 2869, 2022.
- [6] G. B. J. Mdemaya, D. Machuve, and N. N. Mketo, “Efficient Load-Balancing and Container Deployment for Enhancing Latency in Edge IoT Networks,” *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 15, no. 10, pp. 276–286, 2024.
- [7] P. S. Prashanth, M. S. Raunak, and S. C. Sharma, “DL³: Adaptive Load Balancing for Latency-Critical Edge Cloud Applications,” in *Proc. Int. Conf. Network and Service Management (CNSM)*, 2024, pp. 1–9.
- [8] B. Wydrowski, et al., “Load is Not What You Should Balance: Introducing Prequal,” in *Proc. USENIX Symposium on Networked Systems Design and Implementation (NSDI '24)*, 2024, pp. 1–15.
- [9] H. Saokar, M. Chowdhury, S. Somasundaram, et al., “ServiceRouter: Hyperscale and Minimal Cost Service Mesh at Meta,” in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI '23)*, 2023, pp. 969–985.
- [10] Pomerium, “Kubernetes Compliance: NIST, CIS & PCI – Actionable Guide,” Pomerium Blog, 2024. [Online]. Available: <https://www.pomerium.com/blog/kubernetes-compliance-nist-cis-pci-actionable-guide>
- [11] Cloud Security Alliance (CSA), “PCI DSS Pointers for Kubernetes,” CSA Report, 2023. [Online]. Available: <https://circle.cloudsecurityalliance.org>